

Reviewer Pack — Minimal Incidence Count Inverse Proportional to Disjointness...

Entry bee7b259537e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 09:07:29 UTC

Minimal Incidence Count Inverse Proportional to Disjointness Communication Complexity

Entry ID: bee7b259537e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 09:07:29 UTC

1. Conjecture

Field A (mathematical branch): Incidence Geometry **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

For a disjointness function f on n variables, the minimal number of incidences between the points and lines in the corresponding incidence structure is inversely proportional to the communication complexity of f , i.e., $\log(\text{communication complexity}) = \Theta(1 / \min_incidence)$.

Rationale (proposer's reasoning):

Incidence geometry provides tools to analyze geometric configurations, which might reveal structural properties of communication matrices that influence their complexity. The inverse proportionality suggests that more structured configurations (higher $\min_incidence$) could imply lower communication complexity.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 64f601fd6bb262ad

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_disjointness_instance(n):
    points = list(range(1, 2**n))
    lines = []
    for i in range(n):
        line = [j for j in points if (j & (1 << i)) != 0]
        lines.append(line)
    return points, lines

def compute_min_incidence(points, lines):
    incidence_matrix = [[0] * len(lines) for _ in range(len(points))]
    for i, point in enumerate(points):
        for j, line in enumerate(lines):
            if any(point & (1 << k) != 0 for k in line):
                incidence_matrix[i][j] = 1
    min_incidence = min(sum(row) for row in incidence_matrix)
    return min_incidence

def compute_communication_complexity(points, lines):
    n = len(points)
    communication_matrix = [[0] * n for _ in range(n)]
    for i, point in enumerate(points):
        for j, point2 in enumerate(points):
            if any(point & (1 << k) != 0 for k in points[j]):
                communication_matrix[i][j] = 1
    # Simplified version of communication complexity calculation
    return sum(sum(row) for row in communication_matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        points, lines = generate_disjointness_instance(n)
```

```

min_incidence = compute_min_incidence(points, lines)
comm_complexity = compute_communication_complexity(points, lines)

if min_incidence == 0:
    return {
        "metric_name": "min_incidence",
        "metric_value": None,
        "instances_tested": n,
        "conjecture_holds": False,
        "counterexample": "min_incidence_zero"
    }

log_comm_complexity = math.log(comm_complexity)
conjecture_holds = abs(log_comm_complexity - (1 / min_incidence)) < 0.1

results.append({
    "n": n,
    "min_incidence": min_incidence,
    "comm_complexity": comm_complexity,
    "log_comm_complexity": log_comm_complexity,
    "conjecture_holds": conjecture_holds
})

return {
    "metric_name": "min_incidence",
    "metric_value": sum(result["min_incidence"] for result in results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": all(result["conjecture_holds"] for result in results),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 200, 7))

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\\"seed\\"}: {seed}, **{result}**}")
        all_results.append(result)

    mean_value = sum(r["metric_value"] for r in all_results if r["metric_value"] is not None) / len(all_results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in all_results if r["metric_value"] is not None) / len(all_results))
    support_fraction = sum(1 for r in all_results if r["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\\"min_incidence_zero\\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f4380202.py", line 88, in <module>
    result = run_trial(seed)
             ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f4380202.py", line 52, in run_trial
    comm_complexity = compute_communication_complexity(points, lines)
                     ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f4380202.py", line 39, in compute_communication_complexity
    if any(point & (1 <= k) != 0 for k in points[j]):
        ~~~~~
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error; unable to evaluate conjecture validity | next: Fix data structure handling in compute_communication_complexity to handle integer inputs

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111503
2	propose	ollama_remote	qwen3:8b	0	0	121218
3	preregistration	ollama_remote	qwen3:8b	0	0	24529
4	novelty	ollama_remote	qwen3:8b	0	0	20649
5	novelty	ollama_remote	qwen3:8b	0	0	15754
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10242
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10482
8	judge	ollama_remote	qwen3:8b	0	0	18023

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 332401 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/bee7b259537e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/bee7b259537e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bee7b259537e.tar.gz` (if generated)